



React Hooks – Session Notes (April 8, 2025)



What Are React Hooks?

- Introduced in React 16.8
- Hooks allow function components to access React features like:
 - State (`useState`)
 - Lifecycle methods (`useEffect`)
 - Context (`useContext`)
- Hooks eliminate the need for class components in most cases.



"Hooks allow us to 'hook' into React features such as state and lifecycle methods."



Rules of Hooks

1.  Hooks can only be called inside React function components.
2.  They must be called at the top level of the component.
3.  Hooks cannot be conditional (no `if`, `for`, or `switch`).
4.  You must import hooks from `react`.



Note: Hooks do not work in class components.



`useState` Hook



Definition:

The `useState` hook allows you to add state to a functional component.

jsx



Copy



Edit

```
const [state, setState] = useState(initialValue);
```

- state : Current state value
- setState : Function to update that state

Usage Example from Slide:

jsx



Copy



Edit

```
import { useState } from "react"; function FavoriteColor() { const [color, setColor] = useState(""); return ( <div> <h1>{color}</h1> <button onClick={() => setColor("blue")}>Change me</button> </div> ); }
```

✓ Explanation:

- We initialize state as an empty string: `useState("")`
- `color` holds current state
- `setColor("blue")` updates the state to "blue"
- Button click triggers state update



Live Code Demo (from today's project):

jsx



Copy



Edit

```
import React, { useState } from 'react'; function Test1() { const [count, setCount] = useState(100); const DecreaseButton = () => { setCount(count - 1); }; return ( <div> <h1>{count}</h1> <button onClick={DecreaseButton}>Decrease</button> </div> ); } export default Test1;
```

This example demonstrates:

- State variable `count` initialized to 100
- Button click decreases the value using `setCount(count - 1)`



Slides Used

1. Intro to React Hooks
 2. Rules of Hooks
 3. What is `useState` and how to initialize it
 4. Example of `useState` with `color`
-



Summary

- Hooks are powerful tools to manage state and side effects in functional components.
- `useState` is the most basic and frequently used Hook.
- Remember to always follow the **Rules of Hooks** for clean and predictable behavior.